

Can Multi-LLM Reasoning Improve Decision-Making?

Evidence from Elastic GPU Cluster Scheduling

Lay Kim Seng

Cyberscience Center, Tohoku University

lay.kim.seng.q4@dc.tohoku.ac.jp

Abstract—A GPU job that can be resized makes two demands on a scheduler: which job to start, and how many devices to give it. Trace-driven studies almost always erase the second decision by replaying each job at its recorded size, and we show that this choice decides the outcome of the study. Replaying the MIT Supercloud accounting log rigidly on an 80-GPU pool, five standard schedulers differ by at most 1.3 percentage points of occupancy, recover on average 0.2% of idle capacity, and separate on no service metric after correction: the comparison is close to vacuous by construction. Restoring elasticity through a malleability-aware simulator makes the same metric controllable and reverses its meaning. Sizing then moves service quality by an order of magnitude more than any ordering rule, and the sizing rule that attains the highest occupancy we measure, 0.988 against 0.890, is simultaneously the worst on every service axis, because sublinear speed-up converts capacity into busy-ness rather than into completed work. On held-out windows the four queue-ordering rules we test span 0.04 percentage points of deadline violations while sizing spans 15.3 to 46.4, and among auction valuations only pricing a job’s deadline pressure matters, 8.6 against 14.9 to 15.3. The advantage of auction pricing over queue ordering is concentrated in one overloaded window of eleven, where violations fall from 91.0% to 21.6% while the median difference is zero. Counterfactual enumeration of every policy from sampled states bounds what any state-dependent selector could add at 2.03 points for queue ordering and 0.07 for auctions, with a median of zero. Asked to select policies from that same observable state, a language model does not approach the bound: on held-out states an 8B model carries 8.70 points of mean regret against the deterministic policy’s 1.05, and emits an illegal action on 11 of 24 states where deterministic code emits none. Low-rank fine-tuning changes nothing, at 8.78 points and the same 11 failures, and role separation into demand and supply analysts does not help either. Reasoning earns its cost elsewhere: on an authored suite of operator-stated exceptions, which no public trace supplies, a debate between reviewers handles 43 of 81 cases against 29 for a single model at an identical call budget ($p = 0.0007$), while on control scenes with nothing to reason about every model arm is worse than the deterministic mechanism. We conclude that numeric rationing belongs to deterministic code, and that a language model belongs where the deciding fact reaches the scheduler as text.

Index Terms—GPU scheduling, large language models, resource allocation, multi-agent debate, cluster traces

I. INTRODUCTION

AI training and inference are the dominant consumers of GPU clusters, and they are *elastic*: a training job can run on more or fewer devices, trading device-hours against completion time. A scheduler on such a workload therefore

makes two decisions per job, not one — *which* job to start, and *how large* to make it.

Almost every published evaluation erases the second decision. A trace records that a job used g devices for t seconds; it does not record what that job would have done on $2g$ or on $g/2$, because the counterfactual was never observed. The standard response is to replay the trace *rigidly*, pinning each job to its historical size. This is usually presented as a conservative simplification. We show it is a suppression, and that it silently invalidates the efficiency claims most commonly made on top of it.

The argument is short. If no job can be resized, the device-seconds a workload consumes are fixed: every scheduler must run the same jobs for the same durations, so it can only move work earlier or later. Utilisation is then close to a conserved quantity, and a comparison of schedulers on it is close to a comparison of nothing. Restoring elasticity does not merely add a tuning knob; it makes utilisation controllable, and — as we show — makes maximising it actively harmful.

Our contributions, all on the MIT Supercloud GPU trace [2] replayed through a malleability-aware batch simulator [8], are:

- 1) **Rigid replay cannot answer an efficiency question** (§IV-B). We quantify how little room a fixed-size replay leaves: occupancy differs by at most 1.3 points across five standard schedulers, recoverable idle capacity averages 0.2%, and no arm separates on service quality. Backfilling is inert on this trace because requested walltimes exceed true runtimes by two to three orders of magnitude.
- 2) **Occupancy is an anti-metric once jobs are elastic** (§IV-C, §IV-D). Sizing moves service quality by an order of magnitude more than ordering does, and the highest-occupancy rule is the worst-service rule in every window we measure. We recommend reporting device-hours per unit of completed work instead.
- 3) **Language-model policy selection does not match deterministic code** (§IV-G, §IV-H). On held-out windows, queue-ordering rules are interchangeable while sizing and the auction’s valuation are not, and the advantage of auction pricing is a single-window tail effect. Counterfactual enumeration bounds the available headroom at 2.03 and 0.07 points with a median of zero, and neither a zero-shot model, a low-rank fine-tune, nor a

demand–supply role split comes near it: eight times the deterministic policy’s regret, and an illegal action on 11 of 24 held-out states.

- 4) **Where a language model does belong** (§IV-I). When the deciding fact reaches the scheduler as operator text, which no public trace carries, a budget-matched debate handles 43 of 81 authored exception cases against 29 for a single model, $p = 0.0007$, with a blind replication and a control showing the same reasoning is harmful where there is nothing to reason about.
- 5) **A protocol that makes such a comparison auditable** (§IV-A). The selector study is frozen before execution: workload-only window selection, a hashed manifest of trace, worlds, code and configuration, a fail-closed gate, and amendments recorded rather than absorbed.

We additionally retain, unchanged, a result that does not depend on any trace (§IV-I): on an authored suite of exceptions expressible only in operator text — which no public GPU trace supplies — a debate between reviewers corrects a single model’s rulings at an identical call budget. That is where a reasoning layer earned its cost in our setting; it is not the numeric rationing decision, and §IV-D explains why the numeric decision has been measured against the wrong target.

II. RELATED WORK

LLM-driven scheduling. LLMsched [1] repairs a single LLM’s proposal with an ILP and reports large gains over Kubernetes. We inherit the reason-then-guarantee spine but invert its default: on numeric rationing the deterministic layer should be the proposer, not just the repairer, and the LLM should be gated behind a contextual trigger.

Learning-based schedulers. DRL schedulers (DeepRM [3], Decima [4]) learn allocation policies but are black-box and brittle under distribution shift. On our interface a tabular Q-learner’s episode-return variance dominates the action effect at realisable sample counts.

Multi-agent LLM structures. Multiagent debate [9] and self-consistency [10] are usually evaluated on QA benchmarks, and LLM-as-judge protocols [11] inherit that setting; budget-matched controls are rare, so a reported gain is rarely separable from the extra calls that buy it. Our text-exception result is budget-matched by construction and includes a blind confirmatory batch.

Malleability and mechanism design. Runtime elasticity is rarely deployed in production [5]; our partial-allocation model rests on it and we state that dependence. Uniform-price auctions are the classical route to truthful revelation, and automated negotiation for resource allocation is a classical subject [12], with prediction-guided *selective* negotiation explored in one-shot fair division [13]; our contribution is measurement in a closed loop containing declarative LLM agents.

III. CONTROLLED POLICY-SELECTOR ARCHITECTURE

Fig. 1 shows the two reasoning structures. At each 1,800-second selection point the model sees available capacity and

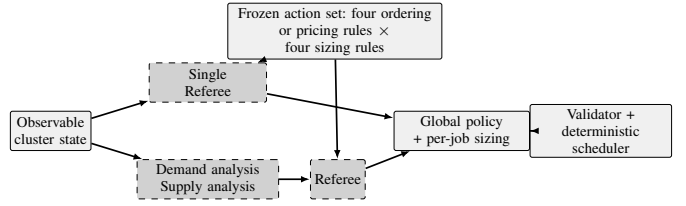


Fig. 1. The controlled 2×2 pipeline. Single and Multi arms receive the same observable state and frozen action menu. Demand and Supply agents produce analysis only; the Referee is the sole decision-maker. All outputs pass through the same deterministic feasibility layer.

scheduler-visible attributes of pending and running jobs: arrival and waiting time, requested walltime and GPU count, priority, deadline pressure, legal malleable bounds, and scaling indicators. It never sees true runtime or future arrivals.

Single Referee. One model returns strict JSON containing one global ordering policy, a default sizing rule, and optional job-specific sizing overrides. The parser expands this into exactly one action for every affected job.

Demand–Supply–Referee. Demand identifies urgent, deadline-pressured, scalable, and shrink-tolerant jobs. Supply describes scarcity, queue pressure, resize cost, and fairness. These agents cannot allocate GPUs; their analyses and the original state are passed to a Referee whose output schema is identical to the Single arm.

The action set. Both structures choose from the same two-part action, and the two policy families are matched in size so that neither referee simply has more options than the other.

The *queue-ordering* family decides which waiting job is considered first: first-come first-served, least-laxity-first on the requested walltime, ageing fair-share on accumulated per-user waiting, and priority-tier ordering that clears the high-priority tier first. The *auction* family replaces that comparison with a single uniform-price clearing and varies only what a job’s claim is priced on: the time it has already waited, its deadline pressure, its owner’s waiting-service deficit, or its priority tier. Holding the clearing rule fixed isolates the valuation, which is the design decision an operator actually makes.

Crossed with either family, four sizing rules decide how many devices a started job receives, and govern later resize points as well. *Requested size* gives a job exactly what its owner asked for and makes it wait until that much is free; this reproduces the rigid replay and is the control. *Queue-proportional* sizing shares the free pool across the waiting queue, so a job may start smaller when others are waiting and grow when they are not. *Maximum-fit* sizing gives every job the largest legal size that fits at the moment it starts. *Benefit-gated* sizing changes a running job’s size only when the estimated gain, measured in seconds against the longest current wait and charged for the extra runtime that shrinking causes, exceeds a fixed threshold of 300 s; its estimates use requested walltimes only, never true runtimes.

A shared validator rejects malformed actions, enforces the 80-GPU capacity and each job’s legal bounds, and invokes a counted deterministic fallback, so the comparison changes

only the policy family and the reasoning structure.

IV. EVALUATION

Setup. We replay the MIT Supercloud GPU trace [2] (a 235-day Slurm accounting log) through ElastiSim [8], a batch-system simulator for malleable workloads. One simulated node is one GPU, so a job that used g GPUs becomes a g -node job; its work is set so that it reproduces its recorded runtime exactly. The scheduler sees each job’s arrival, size and *requested* walltime, never its true runtime — the same information a production batch system has. Capacity is a fixed 80-GPU pool, so a window’s offered load is a property we *measure* rather than a consequence of sizing the cluster to fit it.

Window selection is pre-registered. Every 24 h window of the trace is described by workload statistics alone — job count, GPU-hours, offered load, high-priority fraction, median and p90 runtime, size distribution — with no scheduler executed. Of 5,613 hourly starts, 4,312 carry at least 100 jobs. These are stratified into four offered-load bands ($0.7\text{--}1.0$, $1.0\text{--}1.5$, $1.5\text{--}2.5$, $>2.5\times$) crossed with three high-priority bands ($<2\%$, $2\text{--}10\%$, $>10\%$), and one window is drawn per cell under a fixed seed, subject to a non-overlap constraint. The resulting 12 windows span days 41–222, offered loads $0.75\text{--}4.97\times$ and priority fractions $0.7\text{--}58\%$. Each is preceded by 12 h of warm-up arrivals, which the arm under test also schedules but which are excluded from scoring, so no run begins on an empty cluster. Selection never sees a scheduler result.

Elasticity model. A job may be *moldable*: its size is chosen once, at launch, and fixed thereafter. Speed-up follows Amdahl’s law anchored on the observed point — a job seen at g GPUs reproduces its recorded runtime exactly at g , so the rigid replay is this same model evaluated at the size the job really ran, and the parallel fraction s is swept rather than assumed ($s=1$ is rigid at every size, $s=0$ is linear speed-up). Results below use $s=0.7$, a deliberately conservative value: scaling is sublinear, so growing a job costs more device-time than it saves. We report SLA- k (turnaround exceeding $k\times$ true runtime) at $k \in \{2, 5, 10\}$, mean waiting time, device-hours actually spent, walltime kills, and occupancy over the measured window.

A. Frozen factorial protocol

The core experiment uses 48 non-overlapping 24-hour windows, each with 12 hours of warm-up, selected from workload characteristics before scheduling outcomes are inspected. They are frozen as 24 train, 12 validation, and 12 test windows. Every scored window contains at least 100 jobs. Jobs are malleable at ten resize points with bounds $1 \leq g_i(t) \leq \min(80, 4g_i^{\text{trace}})$, $s = 0.7$, and no modeled resize overhead. Synthetic deadlines are generated once as $D_i = a_i + W_i^{\text{req}}$; a missing walltime receives the fixed 86,400-second estimate. Deadline violation rate is the primary outcome.

The four inferential conditions cross the two reasoning structures with the two policy families. Qwen2.5-14B is held fixed at temperature 0.1 and a 400-token output limit.

Thirty-two deterministic controls exhaustively cross the four policies and four sizing actions in each family. Validation selects the best fixed policy per family; test is reserved for one final report. The inferential unit is the window. The preregistered analysis contains exactly three two-sided paired contrasts on test: market main effect, Multi main effect, and their difference-in-differences interaction. It uses paired- t 95% confidence intervals, exact sign-flip tests, and Holm correction over these contrasts.

Two amendments to the frozen protocol are recorded with it rather than absorbed silently. The first corrects the feasibility assertion, which excluded completed jobs but not killed ones from simultaneous-ownership accounting and so aborted otherwise valid runs; it changes no ordering, sizing, bid, deadline, window, or reported metric. The second reduces the inference seed set from three seeds to one. The multi-agent cell costs 1,703 s per window run against 251 s for the single referee and 8 s for a deterministic control, so the three-seed design needed a further 64 hours of inference. With one response per window, within-window response noise is no longer separable from between-window variance, and at temperature 0.1 a single response is a draw rather than a deterministic answer; the paired tests are therefore more conservative, not less. The protocol plans 864 runs: 768 fixed-policy and 96 LLM-controlled runs over validation and test.

Every result row stores hashes of the processed trace, the generated world, the implementation, the manifest, and the Git revision, and a fail-closed gate re-checks all of them, together with the split, window non-overlap, per-job deadlines, malleable bounds, and speedup-curve anchoring, before any run executes. Superseded hashes are retained explicitly, so rows produced before an amendment remain verifiable instead of being restamped. Runtime assertions enforce cluster capacity, malleable bounds, nonnegative remaining work, and single completion at every scheduler invocation.

B. The rigid replay cannot answer an efficiency question

With every job pinned to its historical size, the device-seconds consumed are fixed by the window, not chosen by the scheduler. Empirically this leaves almost nothing to compete over. Across five schedulers (strict FCFS, first-fit, EASY backfilling, requested-walltime ordering, and a declared/undeclared split) occupancy differs by at most 1.3 points within a window and is identical to three decimal places in several. The idle capacity any ordering rule could still recover — devices free while a job that fitted was waiting — averages 0.2%, with a worst case of 1.3%.

Service quality is equally flat. Over 11 windows no arm separates from the EASY baseline after Holm correction across three SLA thresholds; the largest raw effect, -4.2 points of SLA-2, carries $p_{\text{Holm}} = 0.41$. Two properties of the trace explain the flatness and are worth recording, because both are usually assumed away. First, EASY backfilling [7] is inert here: across 1,351 blocked scheduling points it back-filled 48 times (3.6%), because requested walltimes exceed true runtimes by $100\text{--}600\times$, so no job can be shown to fit

TABLE I
SIZING RULE, 12 PRE-REGISTERED WINDOWS, $s = 0.7$, FIRST-FIT
ORDERING HELD FIXED. MEAN $\pm$ 95% CI.

sizing	device-h	occupancy	SLA-10 (%)	mean wait (s)
Requested-size sizing	3653	0.890	22.3	38,072
Queue-proportional sizing	3339	0.964	32.5	39,756
Maximum-fit sizing	9581	0.988	76.2	270,522

before the head job’s reservation. EASY and strict FCFS are byte-identical on most windows. Second, requested walltime carries no runtime signal: among 487 jobs in one window there are 9 distinct requested values, and the rank correlation with true runtime is $+0.16$, -0.11 , -0.73 and -0.01 across four windows — the only significant value is negative. Any “shortest-job-first” arm on this trace is therefore ordering on noise.

C. Made elastic, sizing dominates ordering

We now allow every job to be moldable and hold the ordering rule fixed at first-fit, varying only how each job is sized: requested-size sizing honours the size its owner asked for and waits until that many devices are free; maximum-fit sizing takes the largest legal size that fits; Queue-proportional sizing shares free capacity across the waiting queue. Table I reports means over the 12 pre-registered windows.

The spread is an order of magnitude, on an axis that does not exist in the rigid replay. Paired against requested-size sizing, maximum-fit sizing costs $+5,929 \pm 1,707$ device-hours ($p=3 \times 10^{-5}$), $+232,451 \pm 60,823$ s of mean waiting ($p=10^{-5}$) and $+53.9 \pm 12.3$ points of SLA-10 ($p<10^{-5}$). No ordering rule we measured in §IV-B moves any metric by a comparable amount.

D. Utilisation is an anti-metric

The three sizing rules order identically on occupancy and inversely on service. Maximum-fit sizing attains the *highest* occupancy of any arm we measure — 0.988, against 0.890 for requested-size sizing ($+0.1 \pm 0.1$, $p=0.009$) — while being the worst on every service axis simultaneously. In 12 of 12 windows the highest-occupancy arm is the worst-service arm, and in 10 of 12 the highest-occupancy arm is not the best-service arm.

The mechanism is not subtle, which is what makes the metric dangerous. Under sublinear scaling, a job given $4\times$ the devices at $s=0.7$ finishes about $1.6\times$ sooner while consuming $4\times$ the capacity. Maximum-fit sizing therefore converts the cluster into *busy-ness* without converting it into completed work, and the queue behind it grows. Occupancy records the busy-ness and is blind to the waste: on one low-load window it reads 1.000 for all three sizing rules while their mean waiting times span 21,429 s to 244,781 s.

This is the practical consequence of §IV-B. A rigid replay reports occupancy differences of a fraction of a point and no scheduler can game them; the metric looks harmless because it is inert. Made elastic, the same metric becomes controllable — and the controller that maximises it is the one to avoid.

TABLE II
CORE VALIDATION COMPLETION AT THE AMENDED SINGLE SEED. RAW
MEANS OVER 12 WINDOWS; THE TEST SPLIT CARRIES THE
CONFIRMATORY ESTIMATE AND IS NOT SHOWN.

cell	completed	planned	raw deadline violations (%)
fixed non-market	192	192	23.14
fixed market	192	192	21.79
Single referee, ordering	12	12	11.27
Single referee, auction	12	12	10.70
Role-separated, ordering	12	12	11.93
Role-separated, auction	0	12	—

We therefore recommend reporting device-hours spent per unit of completed work, together with waiting time, and retiring occupancy as a headline number for elastic workloads.

E. Sizing under uncertainty is open, not solved

The obvious fix for maximum-fit sizing is to shrink jobs when the queue is deep. It does not work. Paired over the 12 windows, queue-proportional sizing does not beat requested-size sizing on mean waiting ($+1,684 \pm 10,847$ s, $p=0.77$) or on device-hours (-313 ± 637 , $p=0.36$), and is significantly *worse* on SLA-10 ($+10.2 \pm 8.3$ points, $p=0.035$). It also introduces a failure the other rules do not have: shrinking a job lengthens it, and at lower parallel fractions this pushes long jobs past their requested walltime and gets them killed. We therefore count kills as SLA violations rather than as fast completions, since a killed job otherwise flatters every wait-based metric.

We record one negative result about our own method. On the single most contended window ($4.97\times$ offered load), queue-proportional sizing cut mean waiting by 42% against Requested-size sizing and looked like a clear win. It did not replicate across the pre-registered set. Had the window been chosen after seeing that number, the effect would have been reported as a finding. This is the concrete argument for selecting windows from workload statistics alone.

What remains open is therefore the interesting decision: sizing a job under genuine uncertainty about its runtime, on a trace where the only available estimate is uninformative (§IV-B). We therefore test that decision with the controlled policy selectors below.

F. Selector conditions: what the completed runs show

The deterministic half of the factorial is complete on validation and all but two windows of test; the four selector conditions are still executing at the single amended repetition, so no test-side factorial estimate is reported here and RQ1–RQ3 remain open. What the finished deterministic half already settles is the baseline the selectors must beat, and it settles it in a way that matters more than the eventual contrast: the fixed policy is chosen honestly, and choosing it honestly costs almost nothing. Table II reports validation completion for provenance.

Three of the four conditions are complete on the validation split. Paired over its twelve windows, moving the single referee from queue ordering to auction pricing changes deadline violations by -0.57 percentage points (95% CI $[-1.69, 0.56]$; two-

TABLE III

DETERMINISTIC CONTROLS ON HELD-OUT TEST WINDOWS, PAIRED OVER THE 11 WINDOWS ALL 32 CONFIGURATIONS COMPLETED. EACH FAMILY’S ORDERING RULES ARE SHOWN AT QUEUE-PROPORTIONAL SIZING; THE SIZING COLUMN VARIES SIZING UNDER THAT FAMILY’S BEST ORDERING.

family	ordering (at queue-proportional)	viol. (%)	sizing	viol. (%)
non-market	FCFS	15.23	queue-proportional	15.27
	least laxity	15.23	requested size	15.42
	tier FCFS	15.25	benefit-gated	15.99
	fair-share	15.27	maximum-fit	46.35
market	deadline	8.62	queue-proportional	8.62
	priority	14.90	benefit-gated	8.71
	wait	15.23	requested size	8.76
	fair-share	15.28	maximum-fit	60.35

sided exact sign-flip $p = 0.438$), with seven windows tied to the digit, three favouring auctions and two favouring ordering. Separating the referee into demand and supply analysts costs +0.67 points against the single referee on the same family (95% CI $[-0.37, 1.71]$, $p = 0.250$), again with seven ties and one window favouring the role-separated structure. Neither difference supports a claim. With seven of twelve windows identical, twelve windows cannot separate these conditions, and the sign of the role-separation difference is the opposite of the architectural hypothesis. These are validation observations; the confirmatory contrasts belong to the reserved split.

One measurement trap is worth stating because it is easy to repeat and we fell into it first. Averaging each condition over whatever repetitions had finished, thirty-six runs for the single-referee conditions against twenty-one for the role-separated one, placed role separation 0.81 points *ahead*. Restricting to the one repetition every window shares reverses the sign. The same hazard applies to the deterministic controls: the best queue-ordering configuration reads 8.08% on the nine test windows that finished first and 15.27% once the eleventh arrives, because windows differ by an order of magnitude in difficulty. Every comparison in this paper is therefore paired over a window set that all compared configurations have completed, and that set’s size is stated.

Validation selects ageing fair-share with queue-proportional sizing at 10.49% violations for the ordering family, and the deadline-priced auction with queue-proportional sizing at 9.87% for the auction family, each by validation violations alone with a deterministic tie-break, and each carried unchanged to the reserved split. We do not draw a selector-against-fixed-policy comparison here: assembling it from one completed split is exactly the unbalanced reading described above, and §V sets out what would settle it.

G. Inside the action space: ordering is inert, valuation is not

The 32 deterministic controls are complete on validation and on 11 of the 12 test windows, which is enough to characterise the action space the selectors choose within, independently of any selector. Table III reports the held-out test means, paired over those 11 windows.

Two facts constrain what any selector could contribute. First, *ordering is inert*: the four non-market orderings span 15.23 to 15.27 points, a spread of 0.04, so on this workload

which queue order a referee names cannot matter. Sizing under the same ordering spans 15.27 to 46.35. This reproduces §IV-C inside the larger 8×4 space, on held-out windows, with deadline violations rather than SLA-10 as the outcome. Second, within the market family the clearing mechanism is held fixed and only what the bid *values* moves the outcome: valuing deadline pressure gives 8.62 points against 14.90–15.28 for priority, wait, and fair-share valuations. The decision that matters is therefore not the ordering rule but how much capacity each job gets and what its claim is priced on.

The family-level difference is a tail effect and has to be reported as one. The best market configuration beats the best non-market configuration by 6.61 points on average, while the median difference over the same 11 windows is exactly zero: seven windows tie to the digit, three move by at most 1.6 points, and one window moves by 69.4, from 91.0% violations to 21.6%. That window is the only one in the set where the cluster is overwhelmed, and it is the only one where any of these choices is worth much. A mean-only reading would present a one-window rescue as a general property of market clearing.

One test window is excluded from Table III. Under queue-proportional sizing on that window the simulator reproducibly allocates 81 devices from an 80-device pool, and the runtime capacity assertion aborts the run; six of the 32 configurations are affected, all of them under queue-proportional sizing. The same assertion is what discards four oracle states in §IV-H. We report the exclusion rather than loosening the assertion, but it is a defect in the sizing path that the winning configuration in both families happens to exercise, and it is the first thing to fix before these numbers are final.

H. Counterfactual oracle and LoRA protocol

To test whether supervision rather than architecture is the bottleneck, we clone sampled simulator states and execute all 16 policy–sizing actions in the relevant family from the same state, with the same future arrivals and seed. The lowest deadline-violation action supplies an offline label; future outcomes are never included in the online prompt. We sample two states per window and family. Four states from one training window are discarded because a simulator capacity assertion reproducibly reports 81 allocations from an 80-GPU pool. The resulting 1,092 auditable rollouts form 44 complete training examples and 24 validation examples, with disjoint windows and no oracle-only fields in model input.

The median oracle gain is zero in every row of Table IV: selection matters in a minority of states, especially in the non-market family, while a strong fixed market policy leaves almost no mean headroom. We fine-tune Qwen3-8B with rank-16 LoRA on assistant spans only and judge it on the 24 held-out states by valid-action rate, oracle-action agreement, mean counterfactual regret, and improvement over the best fixed training action. This evaluation is deliberately offline: it determines whether the model learned the selector before another expensive closed-loop sweep is warranted.

TABLE IV

COUNTERFACTUAL ORACLE CEILING. GAIN IS THE PAIRED REDUCTION IN DEADLINE VIOLATIONS RELATIVE TO AGEING FAIR-SHARE WITH QUEUE-PROPORTIONAL SIZING (NM) OR DEADLINE-PRICED AUCTION WITH QUEUE-PROPORTIONAL SIZING (MKT).

split	family	n	fixed (%)	oracle (%)	gain (pp)
train	NM	22	17.04	14.73	2.31
train	MKT	22	12.77	12.26	0.51
validation	NM	12	7.57	5.54	2.03
validation	MKT	12	4.85	4.78	0.07

TABLE V

LANGUAGE-MODEL SELECTION AGAINST DETERMINISTIC POLICY SELECTION, ON THE 24 HELD-OUT STATES. REGRET IS THE DEADLINE-VIOLATION SHORTFALL AGAINST THE COUNTERFACTUAL BEST ACTION IN THE SAME STATE; LOWER IS BETTER. “NEAR-OPTIMAL” COUNTS STATES WITHIN 0.1 POINTS OF THAT ACTION.

selector	mean regret	median	near-optimal	invalid
deterministic policy (fixed per family)	1.05	0.0	75.0%	0/24
Qwen3-8B, zero-shot	8.70	0.8	45.8%	11/24
Qwen3-8B, LoRA fine-tuned	8.78	0.8	41.7%	11/24

It did not. Table V places both model arms beside the deterministic policy on the same 24 held-out states.

Neither language-model arm is competitive. Both carry roughly eight times the deterministic policy’s regret, both leave a median state 0.8 points short of the best available action where the deterministic policy leaves it exactly at the optimum, and both emit an unparseable or illegal action on 11 of 24 states, which the deterministic policy never does. Fine-tuning moves nothing: mean regret is marginally worse than zero-shot, the median is identical, near-optimal states fall, and the invalid count is unchanged. An initial reasoning-enabled decode was worse still, at 23 of 24 invalid, so both reported arms use the non-thinking template for this strict-JSON task; invalid outputs are conservatively scored as the worst candidate action.

The closed-loop numbers agree with the offline screening. On the validation split the single referee reaches 11.27% deadline violations on the ordering family and 10.70% on the auction family, and the role-separated referee 11.93% on the ordering family, against 10.49% and 9.87% for the deterministic policies selected on that same split. Those fixed policies were chosen on these windows, so this is not a held-out comparison and we do not test it; the held-out evidence is Table V, and it points the same way. Taken together, a small language model asked to choose scheduling policies from observable state — zero-shot or fine-tuned, alone or split into demand and supply roles — does not match a single well-chosen deterministic policy on this workload, and a closed-loop fine-tuned sweep is not warranted by this screening.

This is a statement about the numeric rationing decision, not about language models in schedulers. The same deterministic policy that dominates here is helpless when the deciding fact is not in the numbers at all, which is the case we turn to next.

TABLE VI

THE TEXT-EXCEPTION SUITE: 81 PRIMARY + 17 CONTROL CASES.

category	n	the exceptional fact arrives as...
nl_policy	21	an operator rule that exists only in words
unmodeled	21	a fact the world model does not represent
corrupt	13	a declaration a note says is wrong
contradiction	10	two statements that cannot both hold
ambiguous	8	an under-specified scene needing judgement
infeasible	8	demand that cannot be met as declared
placebo	9	decorative/expired prose (control)
confirm	8	prose confirming the default (control)

TABLE VII

HANDLED CASES ON THE 81 PRIMARY TEXT EXCEPTIONS (QWEN2.5:14B).

arm	handled	LLM calls
rigid floor (inclusion criterion)	0/81	0
single LLM	27/81	81
single LLM, best-of- N (budget-matched)	29/81	569
debate	43/81	569

I. Where reasoning earns its cost: text exceptions

A deterministic mechanism cannot act on an exception whose content is *text*: an operator instruction, a declaration a note says is wrong, a crash-restart the numbers do not show. We measure this on a pre-registered suite of hand-authored exception scenes.

The suite (Table VI). Each case is a small allocation scene (median two jobs, one supply note, 2–8 free GPUs, ~36 words of prose) with a pre-registered predicate defining the defensible ruling, authored *before* any LLM was run on it. The authoring rule is text-dependence: deleting the exception sentence leaves a scene whose numbers imply a different, perfectly reasonable answer. **Admission required both rigid arms — a fixed ILP over the same submissions and a rule-based floor — to fail the predicate; the 0/81 rigid floor in Table VII is therefore an inclusion criterion, not a finding.** What the suite measures is the comparison *among* arms that can read text. Two control categories guard the boundary: *placebo* scenes with decorative or expired prose (including a keyword trap naming a policy the note itself retracts) and *confirm* scenes whose salient prose confirms the default ranking; the fixed-ILP arm and the deterministic market pass 17/17 and 16/17 of these, respectively. Of the 81 primary cases, 50 were authored blind to the per-case outcomes of the earlier under-powered round, and the harness reproduces that round’s head-to-head exactly ($b=6$, $c=1$).

Arms, budget-matched by construction. All arms read an identical structured decision packet. Alongside a single LLM call we run best-of- N sampling given the *same* call budget as the debate, so the only difference between the two expensive arms is how the budget is spent, not how much there is.

The pre-registered primary contrast, debate vs. budget-matched best-of- N : $b=16$, $c=2$, one-sided exact McNemar $p = 0.0007$. Three controls make this hard to explain away:

(i) *budget alone buys nothing* — best-of- N spends $7\times$ the single arm’s calls and gains two cases ($p = 0.25$, ns), while debate spends the identical budget and gains sixteen; (ii) *a blind confirmatory batch reproduces it* — on the 50 cases authored blind to earlier outcomes, debate 29/50 vs. 19/50, $b=11$, $c=1$, $p = 0.0032$; (iii) *the effect respects the boundary* — on the 17 control scenes every LLM arm scores 12/17 against the deterministic market’s 16/17, i.e. reasoning invoked where there is nothing to reason about is actively harmful. An exploratory per-category split (post-hoc, small strata) locates the gain where a fact is *missing* from the model of the world (unmodeled: 13/21 vs. 4/21, $b=9$, $c=0$) rather than where an instruction is present (nl_policy: 8/21 in every LLM arm): debate helps when reviewers must notice an absence, not follow a presence.

J. Why the suite must be authored

The natural objection is that an authored suite proves little about a real workload. It is the only option available. The Supercloud accounting log carries 29 fields — identifiers, timestamps, requested and allocated resources, partition, priority, exit state — and not one of them is free text. No public GPU or HPC trace we surveyed — including the other widely used GPU trace [6] — ships operator-authored exception text; what text exists in this domain is source code and logs, not scheduling rationale.

This is the boundary, and it cuts both ways. A reasoning layer cannot demonstrate value on a channel the data does not contain, which is why §IV-I is measured on an authored suite rather than in simulation. It also means the numeric results of §IV-B–§IV-E are the whole of what this trace can adjudicate, and on those the decisive quantity is a sizing rule, not a reasoning step. Acquiring a workload that carries operator text remains the prerequisite for testing the exception claim in situ.

V. FUTURE WORK

The closed-loop comparison on held-out windows. §IV-H settles the selection question offline, on held-out states, and the closed-loop validation numbers agree with it; what remains is the same comparison on the reserved split, which is executing. Three properties make it sound and we have already implemented them. The deterministic baseline is chosen on validation and reported once on the reserved split; on our windows that discipline is nearly free, costing 0.05 points for the ordering family and nothing for the auction family against the configuration selectable with hindsight, so no eventual gap can be blamed on a weak control. The comparison is paired per window and read at the median as well as the mean, because §IV-G shows a single overwhelmed window carrying a 6.6-point mean difference. And it belongs beside the counterfactual ceiling, since a selector can only be blamed for headroom that exists.

Selection regret rather than end-to-end outcome. Because most states are ties, end-to-end violation rates are an insensitive instrument for the architectural question. Per-state regret against the counterfactual argmax separates a

selector that picks a harmless equivalent from one that picks a damaging action, and it is measurable on the labels we already generate. The same applies to the contended subset: a contention condition defined from observable state alone, such as queued minimum demand exceeding free capacity, partitions the windows without reference to which method won, and it is where the one-window effect above lives.

The sizing path’s capacity defect. The excluded window in §IV-G and the discarded oracle states in §IV-H share one cause, an 81-device allocation from an 80-device pool under queue-proportional sizing. It must be fixed, not asserted around, before any of these numbers is final.

VI. THREATS TO VALIDITY

The factorial run is incomplete. At the amended single repetition, three of the four selector conditions are complete on validation and the role-separated referee on auctions is absent; the 32 deterministic controls are complete on validation and on 11 of 12 test windows, but no LLM-controlled test cell has finished. Every selector number in this paper is therefore a validation-set observation, not the preregistered confirmatory analysis, and we make no market, Multi, or interaction claim. Raw cell means cannot repair missing paired observations. Reducing three seeds to one also merges response noise into the window-level differences, which widens every interval we would eventually report.

The elasticity model is an assumption, and a swept one. No trace records what a job would have done at a different size, so any elastic study must supply that counterfactual. We supply Amdahl’s law anchored on the observed point, which reproduces each job’s recorded runtime exactly at the size it really used, and we sweep the parallel fraction rather than fixing it: $s=1$ recovers the rigid replay at every size and $s=0$ gives linear speed-up. Our conclusions concern the *shape* of the result across that sweep, not a single value. Real training jobs also pay a reconfiguration cost we do not model, which would penalise Maximum-fit sizing further; our claim that occupancy-maximising sizing is harmful is therefore conservative.

One node is one GPU. Supercloud’s TX-GAIA nodes carry two V100s; we model the allocation unit as a device, so intra-node placement effects are absent.

Elasticity differs across experiments. The diagnostic sizing study (§IV-C) is moldable: size is chosen once at launch. The core factorial is malleable at ten fixed resize points. Neither setting models checkpoint, communication, or reconfiguration overhead, so resize-heavy policies receive an optimistic evaluation; we report resize counts and zero modeled overhead explicitly.

SLA- k is an oracle metric and is length-biased. It is defined from a job’s true runtime, which no scheduler knows at submission, so no arm can target it; and because the allowance scales with the job, short jobs violate readily while the longest decile of jobs cannot violate at all within a 24 h window despite waiting longest. We therefore report $k \in \{2, 5, 10\}$ throughout and lead with waiting time and device-hours, which

carry neither defect. The trace supplies no deadline field. The core experiment therefore uses deadlines generated from requested walltime; its primary result concerns that reproducible synthetic SLA, not production deadline compliance.

The oracle dataset is small. It contains 44 training and 24 validation states, and one training window is absent after reproducible capacity-invariant failures. Counterfactual oracle performance is an upper bound under the simulator, not evidence that an online model can identify the action. The LoRA comparison uses one training run and deterministic decoding; its failure is screening evidence against the present recipe, not proof that fine-tuning cannot work with more states, hyperparameter selection, or constrained decoding. It is not a replacement for the frozen test split.

Twelve windows, one per stratum. A difference confined to one cell cannot be separated from that particular window; the strata establish that a result is not confined to one corner of the load/priority space, not that a stratum causes it. §IV-E reports a case where this mattered.

Only completed GPU jobs are replayed, so failed and cancelled work, non-GPU contention, and the cluster’s background load are absent. LLM latency is hidden behind wait time by the gate in the design, but evaluation runs on the simulation clock.

VII. CONCLUSION

How a trace-driven scheduling study handles job size decides what the study can find. Replayed rigidly, the MIT Supercloud workload leaves standard schedulers separated by at most 1.3 points of occupancy and by nothing at all on service quality, so efficiency comparisons drawn on it are comparisons of a conserved quantity. Restoring elasticity makes the same metric controllable, and then maximising it is the wrong objective: the rule that fills the cluster most completely is the worst rule on every service axis we measure, because sublinear speed-up buys busy-ness rather than completed work. For elastic workloads, device-hours per unit of completed work belongs in the headline and occupancy does not.

Within that corrected frame, our title question has a clear answer for numeric rationing: no. On held-out windows the four queue-ordering rules are interchangeable to within 0.04 points while sizing spans a factor of three, among auction valuations only deadline pressure matters, and the advantage of auction pricing over queue ordering is one overwhelmed window in eleven against a median of zero. Counterfactual enumeration bounds what any selector could add at 2.03 points for ordering and 0.07 for auctions, with zero median gain. Asked to choose within that space, an 8B model carries 8.70 points of held-out regret zero-shot and 8.78 after low-rank fine-tuning, against 1.05 for a single deterministic policy, and emits an illegal action on eleven of twenty-four states where the deterministic policy emits none. Splitting the referee into demand and supply analysts does not recover the gap either. Neither more reasoning structure nor supervision closes a gap this wide, because the decision being contested is arithmetic the code already performs correctly.

Reasoning does earn its budget where the deciding fact is absent from the numbers: on authored operator exceptions, a debate between reviewers corrects a single model at an identical call budget. No public GPU trace carries such text, which is both why that result required an authored suite and why it cannot be transferred to online rationing by assertion. The engineering recommendation follows the evidence rather than the architecture: keep a validated deterministic default, count every malformed answer and fallback, and admit a language model only where a held-out regret test and a paired closed-loop comparison beat that default after inference cost. Whether role separation clears that bar on the reserved split is the question our frozen protocol was built to answer, and it remains open.

REFERENCES

- [1] G. Ding, S. Yang, H. Lin, Z. Chen, and J. S. Yang, “LLM-driven adaptive cloud resource scheduling: bridging reasoning intelligence with optimization guarantees,” *IEEE Open J. Comput. Soc.*, 2026.
- [2] S. Samsi et al., “The MIT Supercloud dataset,” *IEEE High Performance Extreme Computing Conf. (HPEC)*, 2021, pp. 1–8.
- [3] H. Mao et al., “Resource management with deep reinforcement learning,” *HotNets*, 2016.
- [4] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” *SIGCOMM*, 2019.
- [5] A. Tarraf et al., “Malleability in modern HPC systems: current experiences, challenges, and future opportunities,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 9, pp. 1551–1564, 2024.
- [6] Q. Weng et al., “MLaaS in the wild: workload analysis and scheduling in large-scale heterogeneous GPU clusters,” *NSDI*, 2022 (cluster-trace-gpu-v2020).
- [7] D. Lifka, “The ANL/IBM SP scheduling system,” *JSSPP*, 1995.
- [8] T. Özden, T. Beringer, A. Mazaheri, H. M. Fard, and F. Wolf, “ElastiSim: a batch-system simulator for malleable workloads,” *ICPP*, 2022, pp. 1–11.
- [9] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” *ICML*, 2024, pp. 11733–11763.
- [10] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” *ICLR*, 2023.
- [11] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” *NeurIPS Datasets and Benchmarks Track*, 2023.
- [12] B. An, “Automated negotiation for complex multi-agent resource allocation,” Ph.D. dissertation, Univ. of Massachusetts Amherst, 2011.
- [13] M. Croitoru, C. Croitoru, and G. Ganesh, “Prediction-based selective negotiation for refining multi-agent resource allocation,” *ICAART*, vol. 1, 2025, pp. 656–662.